Abriendo un archivo

```
archivo = open(nombre_archivo, modo)
```

1. **archivo**
   Es una **variable** que va a almacenar el objeto archivo que devuelve la función **open()**.
   Este objeto se puede usar para **leer, escribir o manipular** el archivo abierto según el modo elegido.
   Se suele llamar **archivo, f, file** o cualquier otro nombre válido.
2. **open()**
   Es una **función incorporada** de Python que se utiliza para **abrir archivos**.
   Al ejecutarse, crea un objeto de archivo que se puede usar para realizar operaciones.
3. **nombre_archivo**
   Es una **cadena de texto (str)** que indica la **ruta y el nombre del archivo** a abrir.
   Ejemplos:

   a. **"datos.txt"**: un archivo en la misma carpeta del script.
   b. **"C:/documentos/datos.csv"**: una ruta absoluta en Windows.
   c. **"../otros/archivo.json"**: una ruta relativa a una carpeta anterior.
4. **modo**
   Es una **cadena (str)** que especifica el **modo de apertura**, es decir, cómo se va a usar el archivo: lectura, escritura, binario, etc.
   Algunos ejemplos comunes son **'r', 'w', 'a', 'rb', 'r+'**, etc.

```
archivo = open("datos.txt", "r")
```

- Se abre el archivo llamado **datos.txt**
- El archivo se abre en **modo lectura** (**"r"**)
- El objeto devuelto por **open()** se guarda en la variable **archivo** para poder usarlo luego (leer su contenido, cerrarlo, etc.)

Cerrando un archivo

Una vez que terminamos de trabajar con un archivo (ya sea que lo leímos, escribimos o modificamos), es **muy importante cerrarlo** utilizando el método **close()**:

```
archivo.close()
```

Cerrar un archivo no es solo una formalidad. Tiene implicancias técnicas y prácticas importantes:

1. **Libera recursos del sistema**
   Cada archivo abierto consume memoria y otros recursos del sistema operativo.
   Si no se cierra, esos recursos no se liberan correctamente.
2. **Evita pérdida de datos**
   Cuando se escribe en un archivo, a veces los datos quedan en una **memoria intermedia (buffer)** antes de guardarse realmente en el disco.
   Si el archivo **no se cierra**, esos datos podrían **no guardarse completamente**, lo que lleva a archivos corruptos o incompletos.
3. **Evita bloqueos en el sistema**
   Algunos sistemas operativos **bloquean un archivo mientras está abierto**, impidiendo que otros programas lo usen.
   Si el archivo no se cierra, puede quedar "ocupado" o inaccesible.
4. **Buena práctica y legibilidad**
   Cerrar archivos cuando ya no se necesitan es parte de una **programación limpia y ordenada**.
   Ayuda a que otros desarrolladores (o vos mismo en el futuro) entiendan claramente dónde empieza y termina el trabajo con archivos.

with

En Python, existe una forma más **segura y conveniente** de trabajar con archivos: usar la instrucción **with**.

```
with open("datos.txt", "r") as archivo:
    for line in archivo:
        print(f"{line}\n")
```

Abre el archivo y **garantiza que se cerrará automáticamente** al finalizar el bloque, **incluso si ocurre un error dentro del mismo**.

1. **Cierre automático del archivo**
   Ya no es necesario llamar a **archivo.close()**. Python se encarga de cerrar el archivo **automáticamente** cuando termina el bloque indentado.
2. **Más seguro**
   Si ocurre una excepción o error mientras se trabaja con el archivo, igual se cerrará correctamente.
   Esto evita pérdida de datos o archivos bloqueados.
3. **Código más limpio y legible**
   Se reduce el número de líneas y deja explícito que el archivo solo se usará dentro de ese bloque.

Escribiendo un archivo

Para escribir en un archivo, primero hay que abrirlo en un modo que permita escritura, como **"w"** (write) o **"a"** (append).

- **Modo "w":**
  Crea un archivo nuevo o **sobrescribe** el archivo existente.
- **Modo "a":**
  Abre el archivo para **agregar** contenido al final sin borrar lo que ya estaba.

Usamos el método **write()** para escribir texto en el archivo abierto.

```
with open("datos.txt", "a") as archivo:
    archivo.write("Hola, este es un texto de ejemplo.\n")
    archivo.write("Otra línea más.\n")
```

- El método **write() no agrega automáticamente un salto de línea**, por eso hay que poner **\n** al final si queremos saltos de línea.
- Se puede llamar a **write()** varias veces para agregar más contenido.
- Cuando se usa el modo **"w"**, si el archivo existía, su contenido previo se elimina.

Método writelines

- Permite escribir **varias líneas a la vez** en un archivo.
- Recibe como argumento un **iterable** (como una lista o tupla) con cadenas de texto.
- **No agrega saltos de línea automáticamente**, por lo que cada cadena debe terminar con **\n** si se quiere una línea nueva.

```
lineas = [
    "Primera línea\n",
    "Segunda línea\n",
    "Tercera línea\n"
]

with open("datos.txt", "a") as archivo:
    archivo.writelines(lineas)
```

Leyendo un archivo

Una vez que un archivo fue abierto en modo de lectura (**"r"** o **"r+"**), se puede acceder a su contenido de diferentes formas utilizando distintos métodos. Python ofrece varios métodos útiles para leer el contenido de un archivo según lo que necesitemos hacer.

read()

- Lee **todo el contenido** del archivo como una **única cadena de texto**.
- Puede recibir un argumento opcional para indicar la **cantidad de caracteres** a leer.
- Si el archivo es muy grande, leerlo completo puede consumir mucha memoria.

```
with open("datos.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)
```

readline()

- Lee **una sola línea** del archivo (hasta encontrar un **\n**).
- Cada vez que se llama, avanza a la siguiente línea posicionando un cursor imaginario en el último lugar donde leyó.

```
with open("datos.txt", "r") as archivo:
    linea_1 = archivo.readline()
    linea_2 = archivo.readline()
```

readlines()

- Lee **todas las líneas** del archivo y las devuelve como una **lista de cadenas**.
- Cada línea es un elemento de la lista e incluye el carácter **\n.**

```
with open("datos.txt", "r") as archivo:
    lineas = archivo.readlines()
    print(lineas)
```